

Model Predictive Control for B-Spline Trajectory Tracking in Omnidirectional Robots

José Pedro Carvalho^{*†}, António Paulo Moreira^{*‡}, A. Pedro Aguiar^{*†}

^{*}Faculty of Engineering, University of Porto (FEUP), 4200-465 Porto, Portugal

[†]SYSTEC-ARISE Research Center for System and Technologies, 4200-465 Porto, Portugal

[‡] INESC TEC—Institute for Systems and Computer Engineering, Technology and Science, Porto, Portugal

Email:{jpcarvalho, amoreira, pedro.aguiar}@fe.up.pt

Abstract—In the field of intelligent autonomous robots, integrating optimization techniques with classical control theory methods for mobile robot control is an increasingly prominent area of research. The combination enhances robots' ability to perform their tasks more efficiently, reliably, and safely. This paper addresses the development of a path and motion planning framework for omnidirectional robots, leveraging B-Splines and Trajectory Tracking with Model Predictive Control. The proposed framework is evaluated through software-in-the-loop tests using two distinct dynamical models and sets of hyperparameters. Final validation is conducted by implementing the framework within a ROS environment and performing field tests on a robotic platform. The results demonstrate that the robot can reliably track trajectories at its actuation limits, and the proposed framework enables the robot to increase its velocity up to 50% when compared to a PID path-following controller.

Index Terms—Model Predictive Control, Optimization, Omnidirectional Robot, B-Splines, Motion Planning.

I. INTRODUCTION

In the past few years, autonomous mobile robots have gathered more attention from the research community due to their ability to perform various complex tasks and missions. One example where these robots are useful is in the context of Industry 4.0 [1], where mobile robots can serve as warehouse workers and transport loads more efficiently and safely. While many robots with different kinematic constraints exist, one of specific interest for this application is the omnidirectional robots due to their holonomic motion constraints, which enable them to be maneuverable and agile in tight and cluttered spaces such as factories and warehouses [2].

One transversal aspect of all mobile robots and their applications is motion planning. In the context of motion planning, our interest is focused on the task of trajectory tracking [3], where the goal is to design closed-loop control laws that make the mobile robot's pose converge to a trajectory parameterized by time. The literature is well-developed on methods for motion control of omnidirectional robots, with methods ranging from the combination of PID and Feedforward signals [4] to fuzzy control [5], Sliding Mode Control [6], and optimization based methods. The optimization approach is one of the most

promising, with works tackling the usage of Model Predictive Control to the problem of trajectory tracking of Laguerre Polynomials [7], trajectory tracking with friction compensation [8], and nonlinear model predictive control [9].

Model Predictive Control (MPC) is an advanced control methodology employed in diverse fields like bioengineering, economics, chemical engineering, and robotics, driven by its capability to effectively manage complex system dynamics, constraints, and objectives [10]. Unlike traditional control methods, MPC leverages a dynamic model of the system to predict its future behavior, which, combined with an optimization framework, enables the generation of optimal control actions while considering constraints such as actuation limits or safety constraints [11]. By formulating control decisions as an optimization problem, MPC can simultaneously optimize multiple objectives, such as minimizing energy consumption or maximizing efficiency, ensuring robust performance despite uncertainties and disturbances. This is particularly useful for systems subject to these conditions, such as mobile robots [12].

In this paper, we will tackle the problem of Motion Planning in Omnidirectional Robots through the lens of Trajectory Tracking Model Predictive Control. To this end, we will first present the path planning problem and transform it into a trajectory-tracking problem by leveraging B-Splines as a parameterized trajectory. Then, we will formulate a Model Predictive Controller that tracks the trajectory based on the robot's kinematic and dynamic constraints.

The remainder of the paper is organized as follows: Section II tackles the problem of Path and Motion Planning and the generation of adequate reference signals for MPC. Section III introduces possibilities of Kinodynamic models for the Omnidirectional Robot. Section IV contains the formulation of Model Predictive Control for the proposed combination of trajectory tracking and the omnidirectional robot. Finally, Section V presents both the software in the loop and field tests results of the proposed motion planning framework and Section VI contains the main conclusions from this work.

II. PATH AND MOTION PLANNING

For this work, we will consider the context of the Robot@Factory 4.0 Competition. This competition intends to replicate a real-world factory floor with mobile robotic operators. While the specificities of the competition and its

This work was partially supported by project RELIABLE (PTDC/EEI-AUT/3522/2020), R&D Unit SYSTEC - Base (UIDB001472020) and Programmatic (UIDP001472020) funds - and Associate Laboratory Advanced Production and Intelligent Systems ARISE - LAP01122020, funded by national funds through the FCT/MCTES (PIDDAC).

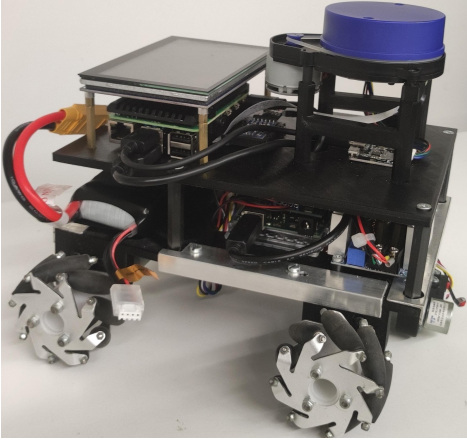


Fig. 1. Four-wheeled omnidirectional platform used in this work [13].

ruleset is out of the scope of this paper, we will use both the robot from Sousa *et. al.* [13] and the competition map. The robot can be visualized in Figure 1 and the Map in Figure 2.

A. Path Planning

Considering the Map in Figure 2, let its representation be defined by a bi-directional graph $\mathcal{M}$ with a set of vertices $\mathbf{V} = \{v_0, \dots, v_j, \dots, v_{65}\}$ and a set of bi-directional edges $\mathbf{E} = \{e_{0,1}, \dots, e_{1,2}, \dots, e_{26,65}\}$. All the vertices are defined by a set of Cartesian coordinates in the global reference frame and an ID, such that $v_j = [x_j, y_j]^T, v_j \in \mathbb{R}^2$, where j is the vertex ID. For the competition, the criterion for choosing the vertices was to assign a vertex to the center of every ArUco and keep its ID.

The edges were selected manually by connecting all the vertices close to each other, also taking into account the walls of the machines and warehouses on the shop floor.

Since the graph is bi-directional and all vertices are reachable from any position, the proposed path-planning solution is based on Dijkstra's algorithm. Considering the start vertex v_s and the destination vertex v_e , the Dijkstra's algorithm can be seen as function $f_d(v_s, v_e) : \mathbf{V} \times \mathbf{V} \rightarrow \mathbf{V}^n$, that maps the starting and final position into an ordered set $\mathbf{P} = \{v_s, v_1, \dots, v_e\}$ which contains all the vertices contained in the shortest path from $v_s \rightarrow v_e$. In the earlier work [13], the path was constructed interpolating the list of vertices with straight lines, where the paths would resemble the edges visualized in Figure 2. This paper will take a different approach by interpolating the control points using B-Splines [14].

B. B-Splines

To have a smoother trajectory, we will use B-Splines in this work for the interpolation of the control points given by Dijkstra's algorithm. For this end, consider once again a generic set of control points $\mathbf{P} \subset \mathbf{V}$. Let the sets $\mathbf{P}_x$ and $\mathbf{P}_y$ denote the x and y coordinates of the vertices in $\mathbf{P}$, and let $n + 1$ be the size of each set $|\mathbf{P}| = n + 1$, where $p_i \in \mathbb{R}^2, 0 \leq i \leq n$ denotes an individual control point. Then,

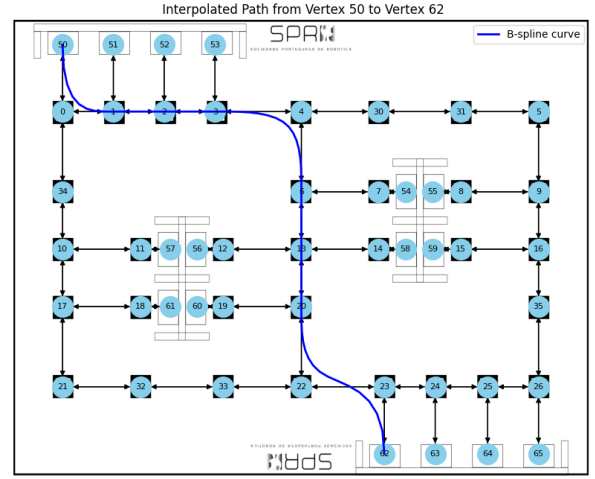


Fig. 2. Robot@Factory 4.0 shop-floor map and respective graph-based representation and an example of interpolating the shortest path from vertex 50 to vertex 62 with a 3rd-order spline.

one can interpolate the set of control points by computing the equation:

$$r(\tau) = \sum_{i=0}^n p_i \cdot B_{i,\kappa}(\tau), \forall \tau_{min} \leq \tau \leq \tau_{max}, \quad (1)$$

where $B_{i,\kappa}$ is a polynomial basis function of order κ , corresponding to a curve of degree of $\kappa - 1$. In order to use equation (1) a knot vector $\tau = [t_0, t_1, \dots, t_{(\kappa-1)+n}]$ where $t_i \leq t_{i+1}, \forall i$, must be specified [14]. This vector determines at which values of τ the curve pieces join. With the knots defined, the basis function can be defined recursively as:

$$B_{i,1}(\tau) = \begin{cases} 1, & t_i \leq \tau < t_{i+1} \\ 0, & \text{otherwise} \end{cases}$$

$$B_{i,\kappa}(\tau) = \frac{\tau - t_i}{t_{i+\kappa-1} - t_i} B_{i,\kappa-1}(\tau) + \frac{t_{i+\kappa} - \tau}{t_{i+\kappa} - t_{i+1}} B_{i+1,\kappa-1}(\tau), \quad (2)$$

and we will define our knot vector to be clamped uniform with $\tau_{min} = 0, \tau_{max} = 1$ such that $\tau \in [0, 1]$. This results in the following knot vector [14]:

$$t_i = \begin{cases} 0, & \text{if } 0 \leq i < \kappa \\ t_{i-1} + \frac{1}{n-\kappa+1}, & \text{if } \kappa \leq i < n \\ 1, & \text{if } i \geq n \end{cases} \quad (3)$$

By setting $\kappa = 3$ and defining the set of control points as $\mathbf{P} = \{50, 1, 2, 3, 6, 13, 20, 22, 23, 62\}$, which represents the shortest path from v_{50} to v_{62} , denoted as $\mathbf{P} = f_d(v_{50}, v_{62})$ one can apply equation (1) to this set, yielding the path that can be visualized in Figure 2.

Note that the spline representation is discrete. To obtain a curve, such as the one shown in Figure 2, we sample the spline equation at multiple values of $\tau \in [0, 1]$. In this work, we discretize the interval $[0, 1]$ into equally spaced samples such that $\mathbb{T} = \{0, l, 2l, \dots, 1\}$, where l is the discretization step.

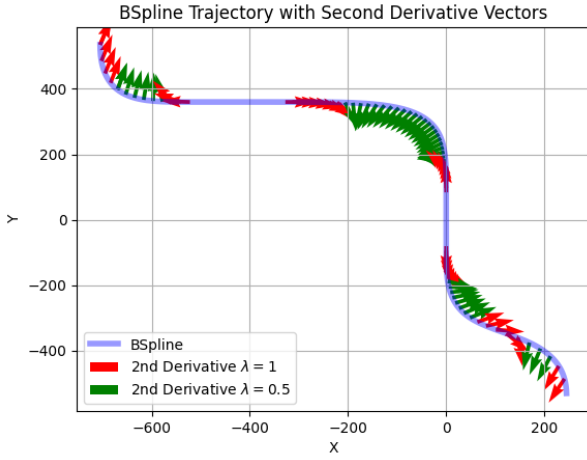


Fig. 3. Example of the Corner Detection technique. The line in blue is the B-Spline path, and the Vectors in green and red are the unit vectors of the evaluated second derivatives. Vectors in red are detected by $\lambda = 1$, whereas vectors in green are detected by any $\lambda \geq 0.5$.

C. Motion Planning

Consider now the standard trajectory tracking problem where, without loss of generality, a generic discrete-time dynamical system $x[k]$ must track a reference signal $r[k]$ such that the tracking error $\|r'[k] - x[k]\|$ tends to zero as $k \rightarrow \infty$. In our case, it would be equivalent to saying that at each instant k , the position of the robot x must match the reference signal r as much as possible. While we already formulated a reference signal for the robot's position in the previous section, this signal is confined in the knot space $\tau \in \mathbb{T}$, whereas the dynamical system exists in the discrete-time domain $k \in \{0, 1, \dots, \infty\}$.

Let our original 2D reference signal $r[\tau]$ be oversampled, such that the distance between consecutive samples of $r[\tau]$ is much smaller than the difference between consecutive samples of the reference signal in time domain $r'[k]$:

$$\|r[\tau + l] - r[\tau]\| \ll \|r'[k + 1] - r'[k]\|, \forall k, \tau \quad (4)$$

Then, by considering the sampling interval T_s of the dynamical system $x[k]$ and the desired velocity at each point v_k , one can construct the time domain reference signal $r'[k]$ by sampling the spline domain signal $r[\tau]$ by following Algorithm 1.

The velocity v_k can either be constant during the whole trajectory or can vary depending on the robot's position. A natural way to choose the velocity would be to put it at a maximum value whenever the trajectory is similar to a straight line and reduce the velocity whenever there is a curve. With this in mind, we will use the B-spline derivative properties to determine whether the trajectory is a straight line. Consider once again the piecewise continuous signal $r(\tau)$ given by equation (1). Its j th-derivative is defined as:

$$r^{(j)}(\tau) = \sum_{i=0}^n p_{x,i} \cdot B_{i,\kappa}^{(j)}(\tau), \forall \tau_{min} \leq \tau \leq \tau_{max}, \quad (5)$$

where the basis function derivative is:

$$B_{i,\kappa}^{(j)}(\tau) = \kappa \left(\frac{B_{i,\kappa-1}^{(j-1)}(\tau)}{t_{i+\kappa} - t_i} - \frac{B_{i+1,\kappa-1}^{(j-1)}(\tau)}{t_{i+\kappa+1} - t_{i+1}} \right). \quad (6)$$

By computing the first and second derivatives of the reference signal, one can obtain the gradient $\vec{\nabla}r = [r'_x, r'_y]^T$ and the Hessian Matrix $\nabla^2 r = \text{diag}(r''_x, r''_y)$, which for this case is diagonal, therefore in the subsequent discussion, we represent it as $\vec{\nabla}^2 r = [r''_x, r''_y]^T$. One of the B-Spline properties is that the derivative of the k th order curve is also a B-Spline of order $k-1$. Since the knot vector is clamped and uniform, this implies that the derivatives of a cubic spline (or higher-order) are continuous up to the second derivative [15]. Thus in our case $\vec{\nabla}^2 r \neq 0$ even if the path is a straight line at that point. In addition, even if we considered straight lines to have a gradient with a zero component, this method becomes too sensitive as the order k of the curve increases. To deal with this problem, we compute the absolute value of the inner product between the two unit vectors:

$$\Theta = \left| \frac{\vec{\nabla}r}{\|\vec{\nabla}r\|} \cdot \frac{\vec{\nabla}^2 r}{\|\vec{\nabla}^2 r\|} \right|. \quad (7)$$

Whenever $\Theta \approx 1$, the vectors have the same direction, which is the situation whenever the path is straight. Thus, we can define a hyperparameter λ , where the condition $\Theta < \lambda$ is true whenever the path has curvature. By defining $\lambda = 1$ the method detects even the most shallow curves, and as λ decreases the minimum steepness required increases. An example of this technique for corner detection with $\lambda = 1$ and $\lambda = 0.5$ can be seen in Figure 3.

Then, the velocity can be updated following the equation:

$$v_{k+1} = \begin{cases} \max(v_k - \delta_v, v_{min}) & \text{if } \Theta < \lambda \\ \min(v_k + \delta_v, v_{max}) & \text{Otherwise,} \end{cases} \quad (8)$$

where δ_v is the acceleration value and v_{min} and v_{max} are the minimum and maximum values of velocity depending on the robot's constraints.

Algorithm 1 Construct Time Domain Reference Signal $r'[k]$

- 1: **Input:** Spline domain signal positions $\mathbf{r}[\tau] = (x_i, y_i)$
- 2: **Input:** Velocity v , sampling interval T_s
- 3: **Output:** Time domain reference signal r'
- 4: **Initialize the trajectory:**
- 5: $\mathbf{r}'[0] \leftarrow \mathbf{r}[0]$
- 6: $k \leftarrow 1$
- 7: **for** $\tau = 0$ to 1 with step l **do**
- 8: $d \leftarrow v \cdot T_s$
- 9: $d_i \leftarrow \|\mathbf{r}'[k-1] - \mathbf{r}[\tau]\|$
- 10: **if** $d_i \geq d$ **then**
- 11: $\mathbf{r}'[k] \leftarrow \mathbf{r}[\tau]$
- 12: $k \leftarrow k + 1$
- 13: **end if**
- 14: **end for**
- 15: **Return:** r'

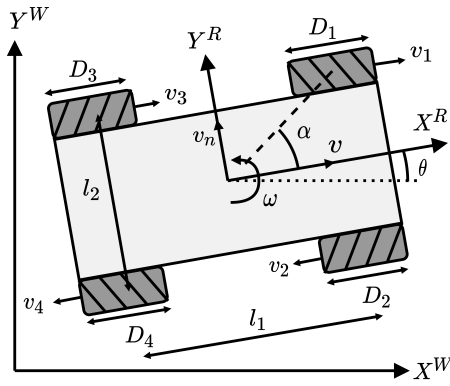


Fig. 4. Four-Wheeled Omnidirectional Robot Kinematics [16].

III. KINODYNAMIC ROBOT MODEL

This work will use a four-wheeled omnidirectional robot as our mobile robotic platform. This section describes both the kinematics and dynamic models for this type of robot.

A. Kinematics

The four-wheeled omnidirectional robot is an interesting mobile platform as its configuration allows holonomic kinematic constraints. This ability to control both pose and orientation in a decoupled manner is useful as it allows for agile and versatile robots. A typical robot with this kinematic configuration can be seen in Figure 4. The robot has four actuators in the form of wheels that allow the control velocity on both axis v, v_n and its angular velocity ω . The relationship between these velocities and the wheel speeds can be derived by the forward kinematics J :

$$J = \begin{bmatrix} \frac{1}{4} & -\frac{1}{4} & \frac{1}{4} & -\frac{1}{4} \\ -\frac{1}{4} & -\frac{1}{4} & -\frac{1}{4} & \frac{1}{4} \\ \frac{1}{2(l_1+l_2)} & \frac{1}{2(l_1+l_2)} & \frac{1}{2(l_1+l_2)} & \frac{1}{2(l_1+l_2)} \end{bmatrix} \quad (9)$$

Then, one can derive the velocities by applying the formula:

$$\begin{bmatrix} v \\ v_n \\ \omega \end{bmatrix} = J \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} \quad (10)$$

B. Dynamic Model

Due to the holonomic motion constraints, movement on the three degrees of freedom is decoupled from another, assuming there is no wheel slip and no actuator saturation. This enables the dynamical model to be approximated by three decoupled integrators:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}, \quad (11)$$

where v_x and v_y are the velocities of the robot in the global referential frame, and ω is its angular velocity. This means that, in this model, the robot is controlled directly by the velocity.

However, it is necessary to transform the velocity from the global referential to the robot referential, which can be done simply by applying a 2D Rotation:

$$\begin{bmatrix} v \\ v_n \end{bmatrix} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} v_x \\ v_y \end{bmatrix}. \quad (12)$$

By multiplying the inverse kinematics matrix J^{-1} , obtained via Moore-Penrose Pseudo-Inverse by the velocities vectors $[v, v_n, \omega]^T$, one can obtain the desired wheel speeds $[v_1, v_2, v_3, v_4]^T$ that the lower-level control system should handle.

Nonetheless, this simplified model assumes that it can apply a velocity instantly, neglecting the effects of acceleration. While, for a given range of speeds and accelerations, these effects are minor for robots with low mass and moment of inertia, they should still be modeled for better accuracy. Consider that for an individual wheel, its angular velocity ω_w is controlled by a closed-loop controller with the approximated transfer function:

$$\frac{W(s)}{R(s)} = \frac{1}{\tau s + 1}, \quad (13)$$

meaning that it is a unitary gain first-order system with a time constant τ . This can be achieved by tuning the lower-level controller, and for this system, a Model Reference Adaptive Control (MRAC) scheme is well suited as we can define a reference model for each motor with the desired time constant τ [17]. Nonetheless, the time-domain differential equation can be written as:

$$\dot{\omega}_w = \frac{r_w(t) - \omega_w(t)}{\tau} \quad (14)$$

Thus, since from equation (10) the robot velocities depend only on the wheel velocities, one can infer that first-order systems also approximate these velocities with a time constant of τ , giving to the augmented integrator model:

$$\dot{\bar{x}} = \begin{bmatrix} 0 & I_3 \\ 0 & -\frac{1}{\tau} I_3 \end{bmatrix} \bar{x} + \begin{bmatrix} 0 \\ \frac{1}{\tau} I_3 \end{bmatrix} \begin{bmatrix} r_{v_x} \\ r_{v_y} \\ r_{\omega} \end{bmatrix}, \quad (15)$$

where $\bar{x} = [x, y, \theta, v_x, v_y, \omega]^T$ and I_n denotes an identity matrix of size n . Instead of controlling directly with the robot velocities in this model, the system input is the reference velocities for the lower-level control system, accounting for the wheel acceleration dynamics.

IV. MODEL PREDICTIVE CONTROL

In this section, we will use the defined kinodynamic models and formulate and solve the Optimal Control Problem (OCP) for Model Predictive Control.

A. Optimal Control Problem

In this work, we will consider the problem of trajectory tracking, where we have a time-varying reference $r_k \in \mathbb{R}^2$ for the desired position at every instant k and the actual robot position $y_k \in \mathbb{R}^2$ such that:

$$\begin{aligned} y_k &= Cx_k \\ e_k &= r_k - y_k, \end{aligned} \quad (16)$$

where $e_k \in \mathbb{R}^2$ is the tracking error vector. We will not consider the yaw of the robot θ to be part of the state or the reference, as due to the kinematic restrictions, this state variable is decoupled from the pose. Furthermore, in most scenarios, we want θ to follow a fixed setpoint, a problem that can be reliably solved by a PID controller while reducing the complexity of the OCP.

For the remainder of the section, we will consider the simplified integrator system represented in equation (11), without the state θ , however, the formulation would be similar for the augmented system of (15). Let the discretized system be described by:

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} T_s & 0 \\ 0 & T_s \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad (17)$$

in which, A is the state transition matrix, B the control input matrix and T_s the sampling time, and finally C the output matrix.

With this discrete system, our unconstrained finite-time OCP can be written as:

$$\begin{aligned} \min \quad & e_N^T S e_N + \frac{1}{2} \sum_{k=0}^{N-1} (e_k^T Q e_k + u_k^T R u_k) \\ \text{Subject to:} \quad & (18) \end{aligned}$$

$$\begin{aligned} x_{k+1} &= A x_k + B u_k \\ e_k &= r_k - C x_k \end{aligned}$$

in which S, Q, R are diagonal positive-definite 2×2 matrices and $N \in \mathbb{N}$ is the prediction horizon. This formulation has the shortcoming that since the robot is controlled in velocity, $u_k \in \mathbb{R}^2$ can never be zero, and therefore, it will be penalized even with perfect tracking. To deal with this issue, consider the following alteration for the system input:

$$u_{k+1} = u_k + \Delta u_k, \quad (19)$$

resulting in u_k being part of the state vector and the input being $\Delta u_k \in \mathbb{R}^2$, which is the change in reference velocity at every instant. Thus, the system is augmented, resulting in the following alterations:

$$\bar{A} = \begin{bmatrix} A & B \\ 0 & I \end{bmatrix}, \quad \bar{B} = \begin{bmatrix} B \\ I \end{bmatrix}, \quad \bar{C} = [C \quad 0], \quad (20)$$

and by substituting the original matrices with the augmented ones in the OCP, we obtain the reference tracking problem that MPC will solve.

B. Closed Form Solution

To efficiently solve the OCP, we will re-write it in matrix form. Thus, consider the vectors:

$$\bar{r} = \begin{bmatrix} r_1 \\ \vdots \\ r_N \end{bmatrix}, \quad \bar{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix}, \quad \bar{\Delta}u = \begin{bmatrix} \Delta u_0 \\ \vdots \\ \Delta u_{N-1} \end{bmatrix} \quad (21)$$

Then, the OCP is rewritten as:

$$\min Z = \frac{1}{2} \bar{x}^T \bar{Q} \bar{x} - \bar{r}^T \bar{T} \bar{x} + \frac{1}{2} \bar{\Delta}u^T \bar{R} \bar{\Delta}u$$

Subject to:

$$\bar{x} = \bar{A} \bar{x} + \begin{bmatrix} B & & \\ & \ddots & \\ & & B \end{bmatrix} \bar{\Delta}u + \begin{bmatrix} \bar{A} \\ 0 \\ \vdots \\ 0 \end{bmatrix} x_0 \quad (22)$$

where :

$$\begin{aligned} \bar{Q} &= \begin{bmatrix} Q & & \\ & Q & \\ & & \ddots \\ & & & S \end{bmatrix}, \quad \bar{T} = \begin{bmatrix} Q\bar{C} & & \\ & Q\bar{C} & \\ & & \ddots \\ & & & S\bar{C} \end{bmatrix} \\ \bar{R} &= \begin{bmatrix} R & & \\ & \ddots & \\ & & R \end{bmatrix}, \quad \bar{A} = \begin{bmatrix} 0 & & & \\ \bar{A} & 0 & & \\ & \bar{A} & 0 & \\ & & \ddots & \ddots \\ & & & \ddots & \ddots \\ & & & & \bar{A} & 0 \end{bmatrix}. \end{aligned} \quad (23)$$

By furthering representing $\bar{x}$ as a expression that depends only on x_0 and $\bar{\Delta}u$ we achieve:

$$\bar{x} = \underbrace{\begin{bmatrix} \bar{B} & 0 & \cdots & 0 \\ \bar{B}\bar{A} & \bar{B} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \bar{B}\bar{A}^{N-1} & \bar{B}\bar{A}^{N-2} & \cdots & \bar{B} \end{bmatrix}}_M \bar{\Delta}u + \underbrace{\begin{bmatrix} \bar{A} \\ \bar{A}^2 \\ \vdots \\ \bar{A}^N \end{bmatrix}}_{\hat{A}} x_0. \quad (24)$$

Substituting $\bar{x}$ by equation (24) in the objective function and ignoring constant terms, we can rewrite it as:

$$\min Z = \frac{1}{2} \bar{\Delta}u^T \underbrace{(M^T \bar{Q} M + \bar{R})}_H \bar{\Delta}u + [x_0^T \quad \bar{r}^T] \underbrace{\begin{bmatrix} \hat{A}^T \bar{Q} M \\ -\bar{T} M \end{bmatrix}}_F \bar{\Delta}u, \quad (25)$$

With the gradient of the objective function computed as:

$$\nabla Z = H \bar{\Delta}u + F^T \begin{bmatrix} x_0 \\ \bar{r} \end{bmatrix}. \quad (26)$$

Since the objective function is convex (quadratic) and the problem is unconstrained, a global minimum is obtained by equalling the gradient to 0:

$$\bar{\Delta}u = -H^{-1} F^T \begin{bmatrix} x_0 \\ \bar{r} \end{bmatrix}. \quad (27)$$

Thus, we obtained the optimal control law. For further computational efficiency, one can compute $-H^{-1} F^T$ offline and use the first $2M$ rows of the resulting matrix to compute $[\Delta u_0, \dots, \Delta u_{M-1}]^T$, where $M \in \mathbb{N}$ is the control horizon.

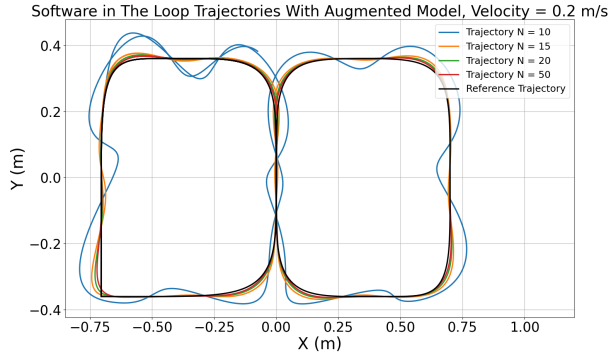


Fig. 5. Sample Trajectories in the Software in the Loop Tests. The model used was the augmented model with a velocity of 0.2m/s.

V. RESULTS

In this section, we will evaluate the motion planning framework in both software in-the-loop scenarios and tests in hardware. The simulations are done in Python, and all code is available in the GitHub repository¹. The simulator simulates the wheel motor dynamics and robot kinematics and can be tuned to fit different robots. The field tests are done in the omnidirectional robot from the work [13]. The localization system is based on an Extended Kalman Filter where wheel odometry and LiDAR data are fused together. The LiDAR detects the position of four beacons placed on the edges of the field; for more details, refer to [13]. The rest of the software stack is the same as in the later work. The main computer is a Raspberry Pi 4B 4GB, and the low-level control loop runs at 50Hz in an Arduino Mega 2560.

A. Software in The Loop

To validate the controller synthesis, the first step was to conduct software in the loop tests. The system is simulated in Python, and both the integrator model of equation (11) and the augmented model described in equation (15) were tested. The cost matrices were manually tuned, resulting in $Q = I_2$ and $R = 5Q$. The tests were made in a benchmark eight-shaped trajectory with constant velocities of $v \in [0.2, 0.4, 0.6]$ m/s, different prediction horizons $N \in [10, 15, 20, 50, 100]$, and the control horizon is set to $M = 1$ in all tests. The tracking

¹<https://github.com/C2SR/MPC-BSpline>
<https://github.com/Jose-PCarvalho/MPC-BSpline-CPP>

TABLE I
ROOT MEAN SQUARE TRACKING ERROR (CM) FOR INTEGRATOR AND AUGMENTED MODELS IN SOFTWARE IN THE LOOP.

N	Integrator Model			Augmented Model		
	0.2 m/s	0.4 m/s	0.6 m/s	0.2 m/s	0.4 m/s	0.6 m/s
10	3.93	11.08	52.50	5.88	38.59	82.03
15	1.07	3.57	17.20	1.52	4.77	35.75
20	0.59	2.07	5.27	0.47	1.64	3.84
50	0.41	1.46	3.13	0.27	0.98	2.01
100	0.45	1.50	3.01	0.25	0.95	1.80

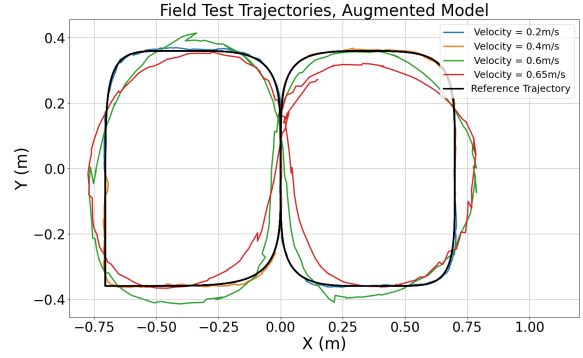


Fig. 6. Field Test trajectories for different nominal velocities, augmented model.

error is defined as the L2-norm $\|x[k] - r[k]\|$, and the mean error (RMSE) value in centimeters for all combinations can be visualized in Table I. Some sample trajectories can be visualized in Figure 5.

From the results, it is possible to infer that the controller is valid and can successfully track the proposed reference trajectories. It is possible to conclude that for good tracking, the prediction horizon N must be at least larger than 20 for all velocities, being that the systems start oscillating at horizons smaller than 10 timesteps and even fail to track at high velocities. It is also possible to infer that the simpler integrator model yields better results than the augmented model when the prediction horizons are smaller. However, it is significantly outperformed by the augmented model when $N > 20$, especially at higher velocities. It is also important to note that there is still some improvement in increasing the prediction horizon from $N = 50$ to $N = 100$, so it should be replicated in the hardware tests.

B. Field Tests

The final validation of the controller is to apply it in real hardware. For this purpose and for real-time performance, the MPC code was ported to C++ and the ROS Framework. The MPC control loop runs at 25Hz, and the prediction horizon N is set to 100, resulting in a 4s lookahead. The rest of the hyperparameters and trajectories are the same as in simulations. The trajectories can be visualized in Figure 6, and in video format². The RMSE for each velocity can be observed in Table II.

The results indicate that the controller can successfully track the references, even considering the uncertainties that were

²https://youtu.be/60KU8g6_VT4

TABLE II
ROOT MEAN SQUARE TRACKING ERROR (CM) FOR FIELD TESTS.

Nominal Velocity	0.2 m/s	0.4 m/s	0.6 m/s	0.65m/s
Integrator Model	0.90	2.26	5.41	14.00
Augmented Model	1.40	3.01	4.45	7.91

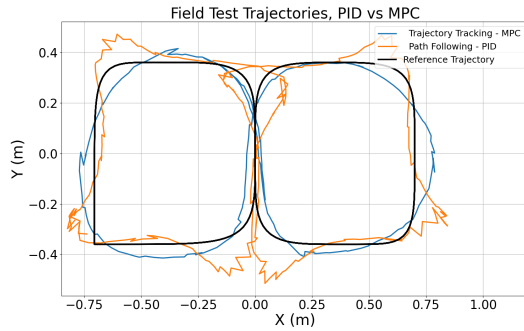


Fig. 7. Comparison between MPC B-Spline Trajectory Tracking and PID Path Following, with nominal velocity 0.6m/s.

not modeled in the simulation, such as wheel slippage and localization error. It is important to note that the maximum robot velocity is approximately 0.69m/s, and therefore, the system can operate at 94% of its capacity, even with this formulation without input constraints. Interestingly, in the hardware tests, the integrator model was shown to perform slightly better at lower velocities, whereas the augmented model yields the best performance at the system's limits. We suspect that these differences are due to nonlinearities introduced by friction and deadzones, which causes the best model in the field tests to vary depending on the operation zone. In simulations these effects are not modeled, making this phenomenon not as evident. Nonetheless, both models' trajectories at lower velocities are very similar and visually almost interchangeable.

Furthermore, in simulation, it was possible to use larger values of Q in order to have more aggressive tracking in the robot; however, in the hardware tests, it frequently caused the wheels to slip and the motors to saturate, causing jittery behavior. Another situation that causes jittery trajectories, as seen in Figure 6, is the localization error and the effect that the EKF update step can have on the robot's estimated pose.

Finally, the previously implemented motion planner was a path-following controller based on PID controllers that followed the edges of the graph [13]. The maximum velocity that the controller could use reliably was 0.4m/s. In a final comparison, a trajectory obtained by the path-following with nominal velocity 0.6m/s is compared to the MPC controller and can be seen in Figure 7. In addition, the path following controller took approximately 12.8s to complete the "lap", whereas the trajectory tracking only took 8.8s.

VI. CONCLUSIONS

In this paper, a Model Predictive Controller scheme that tracks B-Spline trajectories in four-wheeled omnidirectional robots was successfully designed and tested. The controller was shown to be able to function in a real-world scenario where the control law must be computed in real-time and with the uncertainties associated. The robot could track references at its traction and actuation limits while keeping a mean tracking error below 5 cm for velocities below 0.6m/s.

Compared to the robot's previous controller, it was possible to increase the velocity by 50% while keeping the trajectory more stable. Future work will include the improvement of the localization subsystem to enable more aggressive and precise tracking, formulate the constrained optimal control problem, and use data-driven techniques such as Gaussian Process-MPC to account for non-modeled effects such as wheel slippage.

ACKNOWLEDGMENTS

The authors of this work want to thank all the members of team 5dpo for providing both the hardware and existing software stack that made this work possible.

REFERENCES

- [1] S. Vaidya, P. Ambad, and S. Bhosle, "Industry 4.0—a glimpse," *Procedia manufacturing*, vol. 20, pp. 233–238, 2018.
- [2] K. Kanjanawanishkul and A. Zell, "Path following for an omnidirectional mobile robot based on model predictive control," in *2009 IEEE International Conference on Robotics and Automation*, 2009, pp. 3341–3346.
- [3] A. P. Aguiar and J. P. Hespanha, "Trajectory-tracking and path-following of underactuated autonomous vehicles with parametric modeling uncertainty," *IEEE Transactions on Automatic Control*, vol. 52, no. 8, pp. 1362–1379, 2007.
- [4] R. B. Sousa, P. G. Costa, and A. P. Moreira, "A pose control algorithm for omnidirectional robots," in *2021 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*, 2021, pp. 91–96.
- [5] K. Watanabe, "Control of an omnidirectional mobile robot," in *1998 Second International Conference. Knowledge-Based Intelligent Electronic Systems. Proceedings KES'98 (Cat. No.98EX111)*, vol. 1, 1998, pp. 51–60 vol.1.
- [6] X. Fu, S. Wang, J. Yang, Y. Wang, and Z. Liu, "Adaptive sliding mode control for omnidirectional mobile robot based on a new friction modeling," in *2017 International Conference on Computer Technology, Electronics and Communication (ICCTEC)*, 2017, pp. 1042–1047.
- [7] Z. Zeng, H. Lu, and Z. Zheng, "High-speed trajectory tracking based on model predictive control for omni-directional mobile robots," in *2013 25th Chinese Control and Decision Conference (CCDC)*, 2013, pp. 3179–3184.
- [8] J. C. L. Barreto S., A. G. S. Conceição, C. E. T. Dórea, L. Martinez, and E. R. de Pieri, "Design and implementation of model-predictive control with friction compensation on an omnidirectional mobile robot," *IEEE/ASME Transactions on Mechatronics*, vol. 19, no. 2, pp. 467–476, 2014.
- [9] A. S. Conceição, H. P. Oliveira, A. S. Silva, D. Oliveira, and A. P. Moreira, "A nonlinear model predictive control of an omni-directional mobile robot," in *2007 IEEE International Symposium on Industrial Electronics*, 2007, pp. 2161–2166.
- [10] J. Rawlings, D. Mayne, and M. Diehl, *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2017.
- [11] F. Borrelli, A. Bemporad, and M. Morari, *Predictive Control for Linear and Hybrid Systems*. Cambridge University Press, 2017.
- [12] L. Brunke *et al.*, "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annual Reviews in Control*, vol. 5, pp. 411–444, 5 2022.
- [13] R. B. Sousa *et al.*, "A robotic framework for the robot@factory 4.0 competition," in *2024 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*, 2024, pp. 66–73.
- [14] L. Piegel and W. Tiller, *The NURBS Book*, 2nd ed. New York, NY, USA: Springer-Verlag, 1996.
- [15] R. T. Rodrigues, A. P. Aguiar, and A. Pascoal, "A coverage planner for auvs using b-splines," in *2018 IEEE/OES Autonomous Underwater Vehicle Workshop (AUV)*, 2018, pp. 1–6.
- [16] R. B. Sousa, M. R. Petry, P. G. Costa, and A. P. Moreira, "Optiodom: a generic approach for odometry calibration of wheeled mobile robots," *Journal of Intelligent and Robotic Systems: Theory and Applications*, vol. 105, pp. 1–22, 6 2022.
- [17] N. T. Nguyen, *Model-Reference Adaptive Control*. Springer, 2018.